Lab Assignment 21.5

Program	**: B. Tech (CSE)**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: II**
Academic Session	**: 2025-2026**
Name of Student	**: Sruthi**
Enrollment No.	**: 2403A51L10**
Batch No.	**: 51**
Date	**: 10-04-2026**

Task A:

Prompt 1: Generate a simple Python program for a Student Grade Calculator.
It should:
- Take marks of 6 subjects as input
- Calculate total and average
- Display grade based on average

Code:

```python
def calculate_grade(average):
    if average >= 90:
        return 'A'
    elif average >= 80:
        return 'B'
    elif average >= 70:
        return 'C'
    elif average >= 60:
        return 'D'
    else:
        return 'F'
def main():
    marks = []
    for i in range(1,7):
        mark = float(input(f"Enter marks for subject {i}: "))
        marks.append(mark)
    total = sum(marks)
    average = total / len(marks)
    grade = calculate_grade(average)
    print(f"Total Marks: {total}")
    print(f"Average Marks: {average:.2f}")
    print(f"Grade: {grade}")
if __name__ == "__main__":
    main()
```

Output:

```
Enter marks for subject 1: 68
Enter marks for subject 2: 69
Enter marks for subject 3: 89
Enter marks for subject 4: 78
Enter marks for subject 5: 98
Enter marks for subject 6: 65
Total Marks: 467.0
Average Marks: 77.83
Grade: C
```

Prompt 2: Improve the Student Grade Calculator program by:

- Adding input validation (marks between 0–100)
- Handling invalid inputs using loops
- Allowing user to re-enter marks if incorrect
- Adding comments for clarity
- Displaying grade using functions

Code:

```python
def calculate_grade(average):
    """Calculate grade based on average marks."""
    if average >= 90:
        return 'A'
    elif average >= 80:
        return 'B'
    elif average >= 70:
        return 'C'
    elif average >= 60:
        return 'D'
    else:
        return 'F'
def main():
    """Main function to execute the grade calculator."""
    marks = []
    for i in range(1, 6):
        while True:
            try:
                mark = float(input(f"Enter marks for subject {i} (0-100): "))
                if 0 <= mark <= 100:
                    marks.append(mark)
                    break
                else:
                    print("Invalid input. Marks must be between 0 and 100. Please try again.")
            except ValueError:
                print("Invalid input. Please enter a numeric value.")
    total = sum(marks)
    average = total / len(marks)
    grade = calculate_grade(average)
    print(f"Total Marks: {total}")
    print(f"Average Marks: {average:.2f}")
    print(f"Grade: {grade}")
if __name__ == "__main__":
    main()
```

Output:

```
Enter marks for subject 1 (0-100): 78
Enter marks for subject 2 (0-100): 89
Enter marks for subject 3 (0-100): 87
Enter marks for subject 4 (0-100): 98
Enter marks for subject 5 (0-100): 78
Enter marks for subject 6 (0-100): 79
Total Marks: 509.0
Average Marks: 84.83
Grade: B
```

Explanation:

- The original version of the Student Grade Calculator program was a basic implementation that took marks for 6 subjects, calculated the total and average, and displayed the grade based on the average. However, it lacked input validation, which meant that users could enter invalid marks (e.g., negative numbers or numbers greater than 100) without any error handling.
- The improved version of the program addresses these issues by adding input validation to ensure that marks entered are between 0 and 100. It uses a while loop to repeatedly prompt the user until valid input is received. Additionally, it includes error handling for non-numeric inputs using a try-except block.

Task B:

Prompt 1: Create a simple Library Management System in Python.

Code:

```python
def add_book(library):
    """Add a book to the library."""
    title = input("Enter book title: ")
    author = input("Enter book author: ")
    library.append({'title': title, 'author': author})
    print(f"Book '{title}' by {author} added to the library.")
def display_books(library):
    """Display all books in the library."""
    if not library:
        print("No books in the library.")
    else:
        print("Books in the library:")
        for book in library:
            print(f"Title: {book['title']}, Author: {book['author']}")
def main():
    """Main function to execute the library management system."""
    library = []
    while True:
        print("\nLibrary Management System")
        print("1. Add Book")
        print("2. Display Books")
        print("3. Exit")
        choice = input("Enter your choice (1-3): ")
        if choice == '1':
            add_book(library)
        elif choice == '2':
            display_books(library)
        elif choice == '3':
            print("Exiting the program. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 3.")
if __name__ == "__main__":
    main()
```

Output:

```
Library Management System
1. Add Book
2. Display Books
3. Exit
Enter your choice (1-3): 1
Enter book title: Python
Enter book author: Mahesh
Book 'Python' by Mahesh added to the library.

Library Management System
1. Add Book
2. Display Books
3. Exit
Enter your choice (1-3): 2
Books in the library:
Title: Python, Author: Mahesh

Library Management System
1. Add Book
2. Display Books
3. Exit
Enter your choice (1-3): 3
Exiting the program. Goodbye!
```

Prompt2:

Code:

```python
def add_book(library):
    #Add a book to the library.
    title = input("Enter book title: ")
    author = input("Enter book author: ")
    library.append({'title': title, 'author': author})
    print(f"Book '{title}' by {author} added to the library.")
def display_books(library):
    #Display all books in the library.
    if not library:
        print("No books in the library.")
    else:
        print("Books in the library:")
        for book in library:
            print(f"Title: {book['title']}, Author: {book['author']}")
def search_book(library):
    #Search for a book by title.
    title = input("Enter book title to search: ")
    found_books = [book for book in library if book['title'].lower() == title.lower()]
    if found_books:
        print("Book(s) found:")
        for book in found_books:
            print(f"Title: {book['title']}, Author: {book['author']}")
    else:
        print("No book found with that title.")
def return_book(library):
    #Return a book to the library.
    title = input("Enter book title to return: ")
    author = input("Enter book author to return: ")
    library.append({'title': title, 'author': author})
    print(f"Book '{title}' by {author} returned to the library.")
def main():
    #Main function to execute the library management system.
    library = []
    while True:
        print("\nLibrary Management System")
        print("1. Add Book")
        print("2. Display Books")
        print("3. Search Book")
        print("4. Return Book")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")
        if choice == '1':
            add_book(library)
        elif choice == '2':
            display_books(library)
        elif choice == '3':
            search_book(library)
        elif choice == '4':
            return_book(library)
        elif choice == '5':
            print("Exiting the program. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 5.")
if __name__ == "__main__":
    main()
```

Output:

```
Library Management System
1. Add Book
2. Display Books
3. Search Book
4. Return Book
5. Exit
Enter your choice (1-5): 1
Enter book title: Python
Enter book author: Mahesh
Book 'Python' by Mahesh added to the library.

Library Management System
1. Add Book
2. Display Books
3. Search Book
4. Return Book
5. Exit
Enter your choice (1-5): 2
Books in the library:
Title: Python, Author: Mahesh

Library Management System
1. Add Book
2. Display Books
3. Search Book
4. Return Book
5. Exit
Enter your choice (1-5): 3
Enter book title to search: Python
Book(s) found:
Title: Python, Author: Mahesh

Library Management System
1. Add Book
2. Display Books
3. Search Book
4. Return Book
5. Exit
Enter your choice (1-5): 4
Enter book title to return: Python
Enter book author to return: Mahesh
Book 'Python' by Mahesh returned to the library.

Library Management System
1. Add Book
2. Display Books
3. Search Book
4. Return Book
5. Exit
Enter your choice (1-5): 5
Exiting the program. Goodbye!
```

Explanation:

- The first version of the Library Management System allowed users to add books and display the list of books in the library. However, it lacked features for searching for specific books and returning books to the library.
- The second version of the Library Management System introduced several improvements like Search Functionality, Return Book Functionality, Input Validation

Task C:

Prompt 1: Show Git workflow using a simple Python program. Include initial code, Git commands

Code:

```python
def area_of_circle(radius):
    """Calculate the area of a circle given its radius."""
    import math
    return math.pi * radius ** 2
def main():
    radius = float(input("Enter the radius of the circle: "))
    area = area_of_circle(radius)
    print(f"The area of the circle with radius {radius} is: {area:.2f}")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python Sample.py
Enter the radius of the circle: 5
The area of the circle with radius 5.0 is: 78.54
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git init
Reinitialized existing Git repository in C:/Users/BURRA SRUTHI/OneDrive/Desktop/3_2/AI/AI_code/.git/
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git add Sample.py
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git commit -m "Initial commit with area of circle calculation"
[main b3fc9a9] Initial commit with area of circle calculation
 2 files changed, 22 insertions(+), 2 deletions(-)
```

Prompt 2: Rewrite the code that improves the validation and add try and except block

Code:

```python
def area_of_circle(radius):
    """Calculate the area of a circle given its radius."""
    import math
    if radius < 0:
        raise ValueError("Radius cannot be negative.")
    return math.pi * radius ** 2
def main():
    while True:
        try:
            radius = float(input("Enter the radius of the circle: "))
            area = area_of_circle(radius)
            print(f"The area of the circle with radius {radius} is: {area:.2f}")
            break
        except ValueError as e:
            print(f"Invalid input: {e}. Please try again.")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python Sample.py
Enter the radius of the circle: 6
The area of the circle with radius 6.0 is: 113.10
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git add Sample.py
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git commit -m "commit with area of circle calculation"
[main 420df87] commit with area of circle calculation
 1 file changed, 11 insertions(+), 8 deletions(-)
```

Explanation:

The Git workflow for the above code involves the following steps:

1. Initialize a Git repository using `git init`.
2. Add the initial code to the staging area using `git add Sample.py`.
3. Commit the initial code with a message using `git commit -m "Initial commit with area of circle calculation"`.
4. Modify the code to improve validation and add error handling.
5. Add the modified code to the staging area using `git add Sample.py`.
6. Commit the changes with a message using `git commit -m " commit with area of circle calculation"`.

Task D:

Prompt: Create a Python program to demonstrate branching and merging.

First, create a basic program in main branch. Then modify it in a feature branch by adding a new feature (like age input). Show initial code, modified code, and final output.

Code:

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print("Hello,", name)
print("You are", age, "years old")
```

Output:

```
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git init
Reinitialized existing Git repository in C:/Users/BURRA SRUTHI/OneDrive/Desktop/3_2/AI/AI_code/.git/
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git add Task_D.py
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git commit -m "Initial commit"
[main 3630a96] Initial commit
 1 file changed, 6 insertions(+)
 create mode 100644 Task_D.py
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git checkout -b feature/add-age
fatal: a branch named 'feature/add-age' already exists
```

Modified Code:

```
def greet(name, age):
    return f"Hello, {name}! You are {age} years old."
def main():
    name = input("Enter your name: ")
    age = input("Enter your age: ")
    print(greet(name, age))
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git add Task_D.py
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git commit -m "Added age feature"
[main b0e079c] Added age feature
 1 file changed, 4 insertions(+), 1 deletion(-)
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git checkout main
M       Assign_21_5.py
M       Sample.py
Already on 'main'
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> git merge feature/add-age
Auto-merging Task_D.py
CONFLICT (add/add): Merge conflict in Task_D.py
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python Task_D.py
Enter your name: Sweety
Enter your age: 20
Hello, Sweety! You are 20 years old.
```

Explanation:

The commands used in the code is:

- git init
- git add Task_D.py
- git commit -m “Initial Commit”
- git checkout -b feature/add-age
- git checkout main

Task E:

Prompt: Generate professional Git commit messages for given code changes.

Code:

```
def add(a, b):
    """Add two numbers."""
    return a + b
def subtract(a, b):
    """Subtract two numbers."""
    return a - b
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Sum:", add(num1, num2))
print("Difference:", subtract(num1, num2))
# Commit message: "Added basic calculator functions for addition and subtraction, along with user input handling."
```

Output:

```
Enter first number: 20
Enter second number: 30
Sum: 50.0
Difference: -10.0
```

Explanation:

Commit introduces two functions, `add` and `subtract`, which perform basic arithmetic operations. The program also includes user input for two numbers and displays the results of both operations. This enhancement allows users to easily perform calculations directly from the command line.

Task F:

Prompt: Simulate pair programming where Student A writes a basic Python calculator program and Student B improve it using AI by adding functions and error handling. Show both versions and final code with output.

Basic Code:

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b
def main():
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
    print("Sum:", add(num1, num2))
    print("Difference:", subtract(num1, num2))
    print("Product:", multiply(num1, num2))
    print("Quotient:", divide(num1, num2))
if __name__ == "__main__":
    main()
```

Output:

```
Enter first number: 20
Enter second number: 10
Sum: 30.0
Difference: 10.0
Product: 200.0
Quotient: 2.0
```

Improved Code:

```python
def add(a, b):
    """Add two numbers."""
    return a + b
def subtract(a, b):
    """Subtract two numbers."""
    return a - b
def multiply(a, b):
    """Multiply two numbers."""
    return a * b
def divide(a, b):
    """Divide two numbers with error handling for division by zero."""
    if b == 0:
        return "Error: Division by zero is not allowed."
    return a / b
def main():
    while True:
        try:
            num1 = float(input("Enter first number: "))
            num2 = float(input("Enter second number: "))
            print("Sum:", add(num1, num2))
            print("Difference:", subtract(num1, num2))
            print("Product:", multiply(num1, num2))
            print("Quotient:", divide(num1, num2))
            break
        except ValueError:
            print("Invalid input. Please enter numeric values.")
if __name__ == "__main__":
    main()
```

Output:

```
Enter first number: 20
Enter second number: 5
Sum: 25.0
Difference: 15.0
Product: 100.0
Quotient: 4.0
```

Explanation:

- The improved version includes comments for each function, explaining their purpose. The improved version has a while loop that allows the user to re-enter values if they input non-numeric data, using a try-except block to catch ValueError exceptions.
- The basic version does not handle invalid input, while the improved version provides error handling for both non-numeric input and division by zero.

Task G:

Prompt: Simulate collaborative development where Student A creates a login system and Student B creates a registration system using Python. Merge both features into one program and show final output.

Code:

```python
# Login System (Student A)
def login(username, password):
    #Simulate a login system with hardcoded credentials.
    if username == "user" and password == "pass":
        return "Login successful!"
    else:
        return "Invalid username or password."
# Registration System (Student B)
def register(username, password):
    #Simulate a registration system (for demonstration, it just returns a success message).
    return f"User '{username}' registered successfully!"
# Main Program to merge both features
def main():
    while True:
        choice = input("Do you want to (1) Login or (2) Register? Enter 1 or 2: ")
        if choice == '1':
            username = input("Enter username: ")
            password = input("Enter password: ")
            print(login(username, password))
            break
        elif choice == '2':
            username = input("Enter new username: ")
            password = input("Enter new password: ")
            print(register(username, password))
            break
        elif choice == '3':
            print("Exiting the program. Goodbye!")
            break
if __name__ == "__main__":
    main()
```

Output:

```
Do you want to (1) Login or (2) Register? Enter 1 or 2: 2
Enter new username: Sweety
Enter new password: Sweety
User 'Sweety' registered successfully!
```

Explanation:

In this code, we have two main functions: `login` and `register`. The `login` function simulates a login system with hardcoded credentials, while the `register` function simulates a registration system that simply returns a success message. The `main` function allows the user to choose between logging in or registering, and it handles the user input accordingly.

Task H:

Prompt: Simulate a team coding workflow using AI where different modules (login, dashboard, and data storage) are developed separately and then integrated into one Python program. Show final code and output.

Code:

```python
# Login Module (Student A)
def login(username, password):
    #Simulate a login system with hardcoded credentials.
    if username == "user" and password == "pass":
        return "Login successful!"
    else:
        return "Invalid username or password."
# Dashboard Module (Student B)
def dashboard():
    #Simulate a simple dashboard display.
    return "Welcome to your dashboard! Here you can view your data and settings."
# Data Storage Module (Student C)
def store_data(data):
    #Simulate data storage (for demonstration, it just returns a success message).
    return f"Data '{data}' stored successfully!"
# Main Program to integrate all modules
def main():
    while True:
        choice = input("Do you want to (1) Login, (2) View Dashboard, or (3) Store Data? Enter 1, 2, or 3: ")
        if choice == '1':
            username = input("Enter username: ")
            password = input("Enter password: ")
            print(login(username, password))
        elif choice == '2':
            print(dashboard())
        elif choice == '3':
            data = input("Enter data to store: ")
            print(store_data(data))
        elif choice == '4':
            print("Exiting the program. Goodbye!")
            break
if __name__ == "__main__":
    main()
```

Output:

```
Do you want to (1) Login, (2) View Dashboard, or (3) Store Data? Enter 1, 2, or 3: 1
Enter username: user
Enter password: pass
Login successful!
Do you want to (1) Login, (2) View Dashboard, or (3) Store Data? Enter 1, 2, or 3: 2
Welcome to your dashboard! Here you can view your data and settings.
Do you want to (1) Login, (2) View Dashboard, or (3) Store Data? Enter 1, 2, or 3: 3
Enter data to store: Hi
Data 'Hi' stored successfully!
Do you want to (1) Login, (2) View Dashboard, or (3) Store Data? Enter 1, 2, or 3: 4
Exiting the program. Goodbye!
```

Explanation:

In this code, we have three main functions: `login`, `dashboard`, and `store_data`. The `login` function simulates a login system with hardcoded credentials, the `dashboard` function simulates a simple dashboard display, and the `store_data` function simulates data storage by returning a success message. The `main` function allows the user to choose between logging in, viewing the dashboard, or storing data, and it handles the user input accordingly. The program continues to run until the user decides to exit.